



**UNIVERSIDAD
DE ANTIOQUIA**

1 8 0 3

Universidad de Antioquia
Facultad de Ingeniería
Ingeniería de Telecomunicaciones – Región

Desafío I: Sweet Crush

Preinforme de Análisis y Diseño

Asignatura: Informática II
Profesor: Augusto Enrique Salazar Jiménez
Semestre: 2026-2
Estudiantes: Yudis Apiciado
Oscar Lopez

Septiembre de 2026

Índice

1. Contextualización	2
2. Análisis del Problema	2
2.1. Representación y ubicación	2
2.2. Redimensionamiento	2
2.3. Combinaciones y cascadas	3
3. Objetivos	3
4. Diseño de la Solución	3
4.1. Arquitectura modular	3
4.2. Entorno de desarrollo	4
4.3. Acotación del problema	4
4.4. Flujo de juego en consola	5
4.5. Representación de una ficha en memoria	6
4.6. Descripción teórica de las operaciones	6
4.7. Modificación estructural	7
4.8. Visualización en consola	7
5. Plan de Pruebas	7
6. Aspectos ajustables	7
7. Conclusión	8

1. Contextualización

El desafío consiste en implementar el juego *Sweet Crush* en C++, bajo restricciones estrictas de programación estructurada. Lo distintivo es la **representación compacta del tablero**: cada ficha ocupa exactamente 3 bits y el tablero se almacena como una secuencia continua de bits en memoria dinámica. La solución se desarrolla **exclusivamente en consola**, sin framework gráfico, sin `struct`, `class`, `template`, STL ni `string`, usando punteros, arreglos y memoria dinámica.

El entorno de desarrollo elegido es **Qt Creator**, utilizado únicamente como IDE para la edición, compilación y depuración del código C++. Qt Creator no se emplea como framework gráfico: la aplicación final es de consola y no utiliza widgets, señales ni ninguno de los componentes de la biblioteca Qt. La elección de Qt Creator se justifica por su integración con compiladores GNU, su depurador y su gestión modular de proyectos multi-archivo (`.h` y `.cpp`), lo cual se ajusta a las restricciones del desafío.

[ajustable] La configuración específica del proyecto en Qt Creator (tipo de proyecto, archivo `.pro` o `CMakeLists.txt`) podrá ajustarse durante la implementación, siempre que la solución permanezca siendo de consola y no dependa de librerías externas.

2. Análisis del Problema

2.1 Representación y ubicación

Cada ficha se codifica con 3 bits (6 fichas + 2 estados auxiliares). Para un tablero de F filas y C columnas:

- **Índice lógico:** $i = \text{fila} \cdot C + \text{columna}$
- **Bit inicial:** $b = i \cdot 3$
- **Byte:** $\text{byte} = b/8$ **Offset:** $\text{offset} = b \bmod 8$
- **Memoria total:** $\text{bytes} = \lceil (3 \cdot F \cdot C) / 8 \rceil$

Si $\text{offset} \leq 5$, la ficha está completa en un byte; si $\text{offset} = 6$ o 7 , se reparte entre dos bytes.

2.2 Redimensionamiento

Agregar o eliminar filas o columnas **no se hace copiando bytes**: cambia la posición lógica de casi todas las fichas y debe recorrerse **ficha por ficha**. La memoria solo se reduce cuando la ocupación cae **por debajo del 65 %**.

2.3 Combinaciones y cascadas

Una ficha puede pertenecer a una combinación horizontal y vertical al mismo tiempo. Se **separa para detección de eliminación**: primero se marca en un arreglo auxiliar y luego se elimina una sola vez cada posición. Una cascada es la **secuencia de estados** obtenida al repetir detección → eliminación → reorganización → relleno hasta que no haya combinaciones.

3. Objetivos

Objetivo general: Diseñar una solución modular en C++ que implemente *Sweet Crush* en consola, con representación compacta en 3 bits, memoria dinámica y detección correcta de combinaciones y cascadas.

Objetivos específicos:

1. Definir la codificación de 3 bits para fichas y estados auxiliares.
2. Establecer la relación posición – bit – byte – offset.
3. Gestionar la memoria dinámica respetando el tamaño exacto y el umbral del 65 %.
4. Detectar combinaciones horizontales, verticales y simultáneas.
5. Procesar cascadas hasta un estado sin combinaciones.
6. Permitir agregar o eliminar filas y columnas en cualquier posición.
7. Visualizar el tablero en consola en formato fichas y binario.
8. Desarrollar el proyecto en Qt Creator como IDE, sin emplear su biblioteca gráfica.

4. Diseño de la Solución

4.1 Arquitectura modular

Módulo	Responsabilidad
bits	Lectura y escritura de fichas, cálculo de byte y offset
tablero	Creación, destrucción y redimensionamiento de memoria
juego	Combinaciones, cascadas, reorganización y estadísticas
visualización	Menú en consola, tablero en formato fichas y binario

Cuadro 1: Arquitectura modular.

4.2 Entorno de desarrollo

El proyecto se desarrollará en **Qt Creator** utilizando un kit de compilación con **GNU g++** y el depurador **GDB**. El proyecto se configura como una aplicación de consola (sin Qt Widgets ni módulos Qt), lo cual garantiza el cumplimiento de la restricción de no usar librerías externas en el núcleo de la solución. Qt Creator se emplea únicamente por sus ventajas como IDE: edición multi-archivo, compilación integrada, depuración paso a paso y organización clara del proyecto.

4.3 Acotación del problema

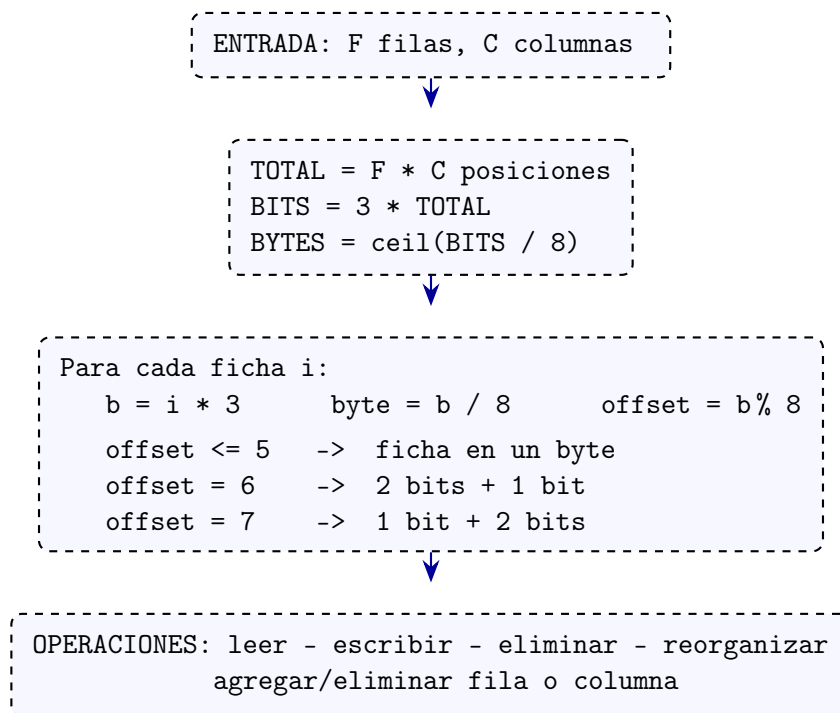


Figura 1: Acotación del problema: de dimensiones a ubicación física en memoria.

4.4 Flujo de juego en consola

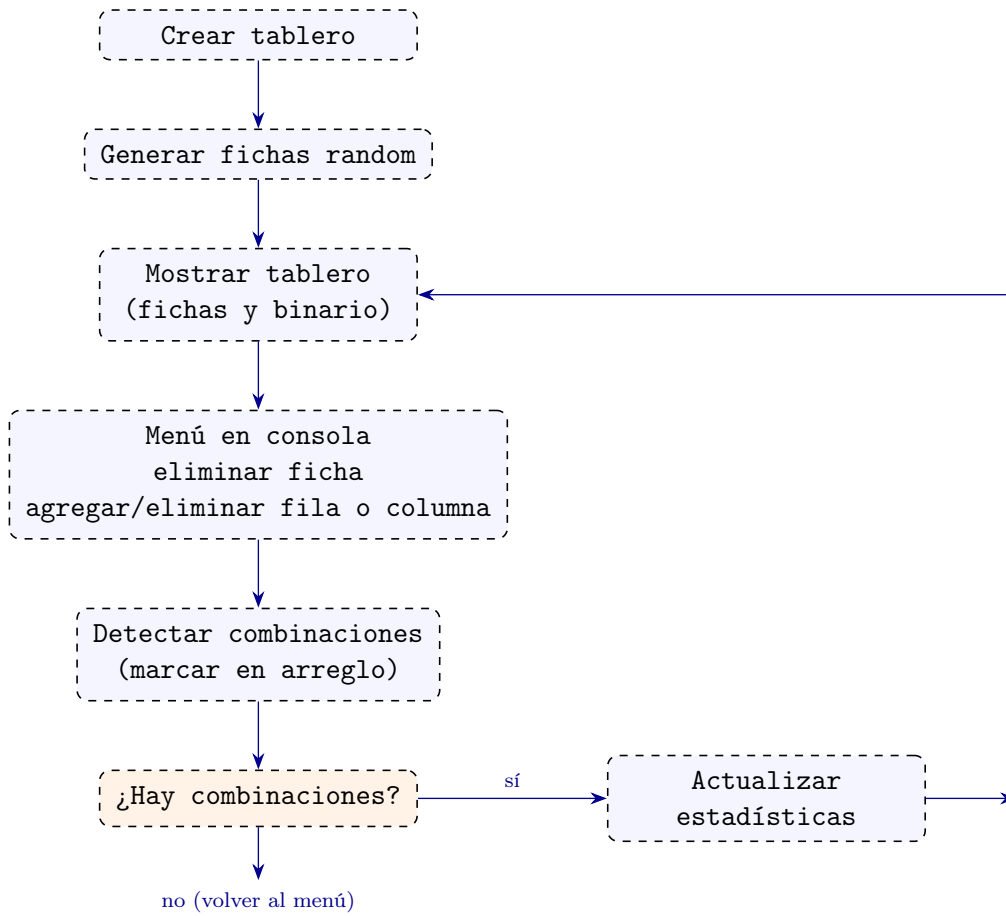
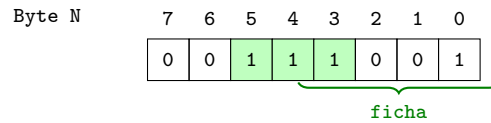


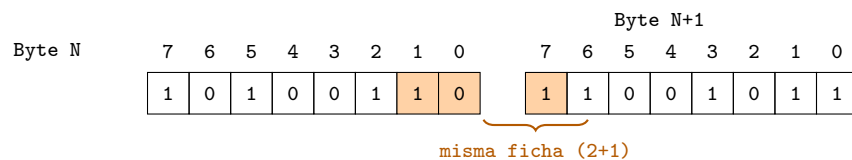
Figura 2: Flujo de juego en consola con cascadas.

4.5 Representación de una ficha en memoria

CASO 1: Ficha completa en un byte (offset < 5)



CASO 2: Ficha repartida entre dos bytes (offset = 6, formato 2+1)



CASO 3: Ficha repartida entre dos bytes (offset = 7, formato 1+2)

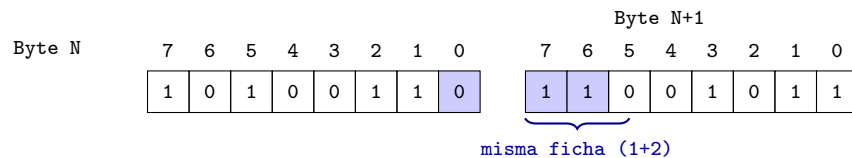


Figura 3: Casos de acceso: ficha en un byte vs. ficha repartida.

4.6 Descripción teórica de las operaciones

Lectura: se localiza byte y offset; si cabe en un byte, se aísla con máscara; si se reparte, se combinan bits de ambos bytes.

Escritura: se protegen bits vecinos, se liberan los 3 bits de la ficha y se deposita el nuevo valor.

Eliminación: se escribe el código de *vacío*, distinto de los 6 códigos de fichas.

Reorganización: las fichas caen al fondo de cada columna, dejando los vacíos arriba.

Relleno: las posiciones vacías se ocupan con fichas aleatorias uniformes.

Detección: se buscan secuencias de 3 o más fichas iguales en horizontal y vertical; cada posición se marca una sola vez en un arreglo auxiliar.

Cascadas: se repite el ciclo hasta que no haya combinaciones.

4.7 Modificación estructural

Para agregar o eliminar filas o columnas en cualquier posición: (1) calcular el nuevo tamaño; (2) recorrer fichas en orden lógico; (3) escribir cada ficha en su nueva posición; (4) generar las nuevas; (5) liberar la memoria anterior. La reducción solo se aplica si la ocupación cae por debajo del 65 %.

4.8 Visualización en consola

Tras cada jugada se muestran dos formatos: **fichas** (un carácter por posición) y **binario** (los 3 bits de cada posición), junto con las estadísticas actualizadas.

5. Plan de Pruebas

Prueba	Objetivo
Lectura/escritura en offsets 0–7	Verificar ambos casos de acceso
Combinación horizontal y vertical simultánea	Confirmar eliminación única
Cascada de tres o más iteraciones	Verificar procesamiento estable
Agregar/eliminar fila y columna intermedias	Confirmar reorganización
Reducción con ocupación < 65 %	Verificar umbral de memoria
Visualización en fichas y binario	Confirmar salida en consola
Compilación y ejecución en Qt Creator	Verificar que el proyecto corre como aplicación de consola

Cuadro 2: Plan de pruebas.

6. Aspectos ajustables

Este preinforme reconoce puntos de flexibilidad que no alteran los pilares del análisis:

- Codificación de fichas (si mejora la legibilidad en consola).
- Número de archivos y distribución de funciones.
- Criterio de puntuación.
- Política exacta de reducción de memoria (respetando el 65 %).
- Caracteres usados para representar las fichas en consola.
- Configuración específica del proyecto en Qt Creator (.pro o CMake).
- Ampliación del conjunto de pruebas.

Los pilares inalterables son: representación en 3 bits, relación posición–byte–offset, separación detección/eliminación, procesamiento iterativo de cascadas y uso de Qt Creator exclusivamente como IDE de consola.

7. Conclusión

El análisis acota el problema en tres pilares: **representación en bits**, **gestión dinámica de memoria** y **lógica de combinaciones simultáneas**. La solución los aborda de forma modular y **exclusivamente en consola**, con estrategias claras para el cruce de bytes, el redimensionamiento ficha por ficha y la eliminación única mediante arreglo de marcado. El desarrollo se realizará en **Qt Creator** utilizado únicamente como IDE, sin depender de su biblioteca gráfica, respetando así las restricciones del desafío. Este preinforme establece la base analítica para la implementación.